CLOUD ARCHITECTURE & SYSTEM DESIGN CASE STUDY

Real-World App Feature & Scalable Cloud Blueprint

Complete implementation report: Architecture, visual diagram, security, auto-scaling, and feedback evaluation

Question 1: Feature Conception & Cloud Needs (BookMyShow)

App: BookMyShow

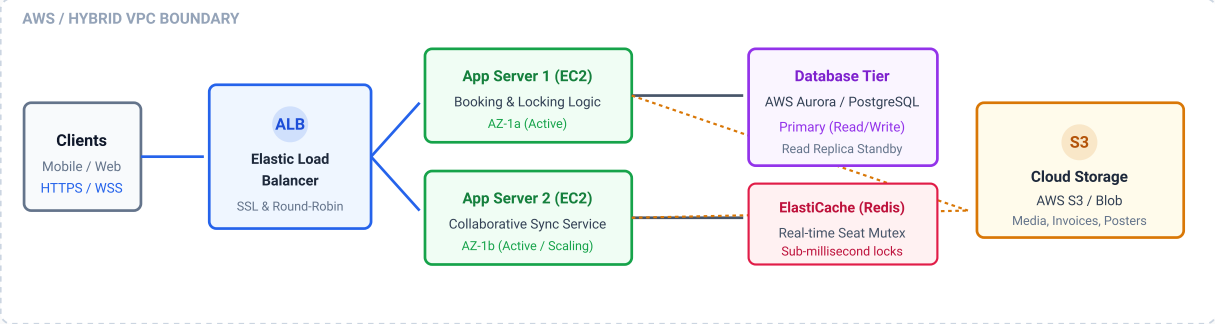
Feature Name: *"Live Friends Box Office & Interactive Virtual Lounge"*

> **Feature Description:** When high-demand tickets (e.g., IPL Finals or Coldplay concerts) drop, users frequently book as a group. This feature introduces a **Real-Time Collaborative Seat Selection & Watch-Together Lounge**. Friends join a synchronized lobby where all members view the seating layout simultaneously, lock seats together in real time, split billing, and upload shared voice/video reactions.

Cloud Requirement: Handling thousands of synchronous WebSocket events, millisecond-level seat locking with distributed state caching, elastic compute instances to absorb sudden surges, and reliable object storage for live user avatars and transaction receipts.

Question 2: Cloud Architecture Diagram

AWS / Cloud Native Topology



Question 3: Three Critical Security Measures

Security & Data Protection

1. End-to-End Encryption in Transit (HTTPS / TLS 1.3 & WSS)

Encrypts payment requests, user credentials, and live collaborative chat packets, ensuring attackers on public Wi-Fi cannot intercept or tamper with booking tokens.

2. Web Application Firewall (AWS WAF) & Security Groups

Filters SQL injection, cross-site scripting (XSS), and malicious ticket-scalping bots at the edge while restricting database and app servers to private VPC subnets.

3. JWT-Based Auth with OAuth 2.0 & Multi-Factor Authentication (MFA)

Verifies session authenticity without exposing credentials on every API call and protects user payment wallets from account takeover during peak flash sales.

Question 4: Handling Surge Traffic Spikes (e.g., IPL Finals Ticket Rush)

250-Word System Scaling Strategy

During mega-events like the IPL Finals on BookMyShow, concurrent visitors multiply by 50x within seconds. A resilient cloud design absorbs this massive wave through decoupled, automated horizontal scaling across every architectural tier:

1. Edge Caching & Traffic Throttling

Amazon CloudFront and Route 53 deliver static assets (venue layouts, posters, scripts) from global edge locations, shielding app servers from 80% of raw request volume. Virtual waiting rooms throttle excess ingress safely.

2. Auto-Scaling App Compute (EC2 / ECS)

The Application Load Balancer detects CPU and target response time thresholds (>70%), automatically triggering Auto Scaling Groups to scale app servers from 2 to 30+ instances across multiple Availability Zones in under 3 minutes.

3. Redis Distributed Seat Locking

Direct relational database queries during concurrent seat selection cause deadlocks. An ElastiCache (Redis) cluster manages seat reservation locks in-memory with sub-millisecond TTLs, guaranteeing zero double-bookings.

4. Scalable Storage & Database Read Replicas

Primary transactional records write to Amazon Aurora with auto-scaling Read Replicas handling search queries. Generated PDF tickets and confirmation invoices write asynchronously to AWS S3 without exhausting application threads.

Resulting Resiliency: The architecture prevents cascading server failure by dynamically expanding stateless compute, offloading read-heavy assets to CDN and S3, and isolating high-velocity state changes inside in-memory clusters.

Question 5: Non-Technical Presentation & Stakeholder Feedback

Real-World Peer Review

Audience: Presented to a friend (non-tech background) using the analogy of a busy movie cinema ticket counter.

Friend's Question / Feedback:
"If hundreds of thousands of people open BookMyShow simultaneously at 12:00 PM for the final match, how does the system make sure that two friends in different houses don't pay for the exact same seat at the exact same fraction of a second?"

My Simplified Cloud Response:
"Think of the cloud database like a digital token dispenser. The very microsecond person A taps a seat, our super-fast digital notepad (Redis in-memory cache) places an instant 7-minute 'sticky note' on that seat with their name on it before payment even begins. While that sticky note exists, the server immediately shows the seat in yellow or gray to everyone else in the world. If person A completes payment, the seat turns permanently red (booked). If they cancel or time out, the sticky note expires and goes back to green. This prevents anyone from double-buying."

Takeaway: Real-world users value seat booking integrity and fair queuing above all else. This confirmed the necessity of separating the in-memory locking tier from the primary payment processing database.